W4 Flask Dashboard + Final Report

Real-time Web Dashboard | API | Map | Day 15-18

4 DAYS

Project Information

Project	Healthcare IoT Deception Honeypot Network
Week	Week 4 of 4 — Dashboard + Completion
Dashboard URL	http://localhost:5000
Backend	Flask + Flask-SocketIO + Gunicorn
Data Source	Elasticsearch via Python elasticsearch client
Stack	Python + Flask + Docker + ELK Stack

Week 4 Objectives

- Create Flask backend with /api/stats, /api/attack_map, /api/recent_attacks, /api/health
- Build dark theme dashboard with Chart.js charts and Leaflet.js world map
- Fix Elasticsearch connection using direct IP instead of hostname
- Add countries count to dashboard stats display
- Generate final threat intelligence report and complete project

DAY 15 Flask Backend — app.py

This setup is used to build and run a containerized dashboard application that fetches attack data from Elasticsearch and displays statistics like total attacks, unique attackers, top IPs, attack types, and countries.

STEP 1 Create Dashboard Dockerfile + app.py

Goal: Backend setup

```
# Dockerfile
cat > dashboard/Dockerfile << 'EOF'
FROM python:3.10-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends gcc && rm -rf
/var/lib/apt/lists/*
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN mkdir -p /app/data
EXPOSE 5000
CMD ["gunicorn","--bind","0.0.0.0:5000","--worker-class","eventlet","-w","1","app:app"]
EOF
cp requirements.txt dashboard/requirements.txt
```

 # app.py

```
#!/usr/bin/env python3
import os, json
from datetime import datetime
from flask import Flask, render_template, jsonify, request
from flask_socketio import SocketIO, emit
from elasticsearch import Elasticsearch

app = Flask(__name__)
app.config["SECRET_KEY"] = os.getenv("SECRET_KEY","dev-secret")
socketio = SocketIO(app, cors_allowed_origins="*")
es = Elasticsearch([os.getenv("ELASTICSEARCH_HOST","http://localhost:9200")])
RANGE_MAP = {"1h":"now-1h","24h":"now-24h","7d":"now-7d","30d":"now-30d"}

@app.route("/")
def index(): return render_template("dashboard.html")

@app.route("/api/stats")
def get_stats():
    tr = RANGE_MAP.get(request.args.get("range","24h"),"now-24h")
    query = {
        "query":{"range":{"@timestamp":{"gte":tr}}},
        "aggs":{
            "total":{"value_count":{"field":"src_ip.keyword"}},
            "unique":{"cardinality":{"field":"src_ip.keyword"}},
            "attack_types":{"terms":{"field":"attack_type.keyword","size":10}},
            "timeline":{"date_histogram":{"field":"@timestamp","fixed_interval":"1h"}},
            "top_ips":{"terms":{"field":"src_ip.keyword","size":20}},
            "top_usernames":{"terms":{"field":"username.keyword","size":10}},
            "countries": {"cardinality": {"field": "geoip.country_name.keyword"}},
        },
        "size":0
    }
    try:
        r = es.search(index="honeypot-*",body=query); a = r["aggregations"]
        return jsonify({
            "total_attacks":a["total"]["value"],
            "unique_attackers":a["unique"]["value"],
```

```
            "attack_types":[{"name":b["key"],"count":b["doc_count"]} for b in
a["attack_types"]["buckets"]],
            "timeline":[{"timestamp":b["key_as_string"],"count":b["doc_count"]} for b in
a["timeline"]["buckets"]],
            "top_ips":[{"ip":b["key"],"count":b["doc_count"]} for b in a["top_ips"]["buckets"]],
            "top_usernames":[{"username":b["key"],"count":b["doc_count"]} for b in
a["top_usernames"]["buckets"]],
            "countries": a["countries"]["value"],


        })
    except Exception as e: return jsonify({"error":str(e)}),500


@app.route("/api/recent_attacks")
def get_recent_attacks():
    query = {"query":{"range":{"@timestamp":{"gte":"now-
1h"}}},"sort":[{"@timestamp":{"order":"desc"}}],"_source":["src_ip","username","password","attack
_type","geoip.country_name","@timestamp"],"size":50}
    try:
        r = es.search(index="honeypot-*",body=query)
        return
jsonify([{"timestamp":s.get("@timestamp",""),"src_ip":s.get("src_ip","Unknown"),"country":s.get("
geoip",{}).get("country_name","Unknown"),"username":s.get("username",""),"password":s.get("passwo
rd",""),"attack_type":s.get("attack_type","unknown")} for hit in r["hits"]["hits"] for s in
[hit["_source"]]])
    except Exception as e: return jsonify({"error":str(e)}),500


@app.route("/api/health")
def health(): return
jsonify({"status":"healthy","timestamp":datetime.now().isoformat(),"elasticsearch":es.ping()})


@socketio.on("connect")
def on_connect(): emit("connected",{"data":"Connected!"})


if __name__ == "__main__":
    socketio.run(app,host="0.0.0.0",port=5000,debug=True)
```

DAY 16 Dashboard HTML Frontend

This code designs the dashboard UI using CSS and dynamically fetches real-time attack statistics from the backend API using JavaScript, then displays total attacks, unique attackers, and countries on the dashboard

STEP 2	Create dashboard.html Dark Theme	Goal: Frontend UI

```
# Create templates dir
mkdir -p dashboard/templates
```

```
# Key CSS
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8"/>
  <title>Healthcare IoT Honeypot Dashboard</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet"/>
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css"/>
  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"/>
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <script src="https://cdn.socket.io/4.5.0/socket.io.min.js"></script>
  <style>
    body{background:#0f1117;color:#e2e8f0;font-family:'Segoe UI',sans-serif}
    .topbar{background:#0b0d14;border-bottom:2px solid #4f8ef7;padding:.75rem 1.5rem;display:flex;align-items:center;justify-content:space-between}
    .brand{font-size:1.1rem;font-weight:700;color:#4f8ef7}
    .live{background:#f74f4f;color:#fff;padding:.25rem .75rem;border-radius:20px;font-size:.75rem;animation:pulse 1.5s infinite}
    @keyframes pulse{0%,100%{opacity:1}50%{opacity:.6}}
    .card2{background:#1a1e2e;border:1px solid #2d3250;border-radius:12px;padding:1.25rem;margin-bottom:1.25rem;border-left:4px solid #4f8ef7}
    .card2.red{border-left-color:#f74f4f}.card2.green{border-left-color:#4ff7a0}.card2.yellow{border-left-color:#f7c94f}
    .num{font-size:2.25rem;font-weight:800;color:#4f8ef7}
    .num.red{color:#f74f4f}.num.green{color:#4ff7a0}
    .lbl{color:#718096;font-size:.8rem;text-transform:uppercase;letter-spacing:1px}
    #map{height:380px;border-radius:10px;border:1px solid #2d3250}
    .feed-item{background:#12151f;border-left:3px solid #f74f4f;border-radius:6px;padding:.65rem 1rem;margin-bottom:.5rem;font-size:.85rem}
    .table{color:#e2e8f0}.table thead th{color:#4f8ef7;border-bottom-color:#2d3250;font-size:.8rem}.table td{border-bottom-color:#2d3250}
    select.rng{background:#1a1e2e;border:1px solid #2d3250;color:#e2e8f0;padding:.35rem 1rem;border-radius:8px}
  </style>
</head>
```

```
<body>
<div class="topbar">
  <span class="brand"><i class="fas fa-shield-virus me-2"></i>Healthcare IoT Honeypot</span>
  <div class="d-flex align-items-center gap-3">
    <span class="live">● LIVE</span>
    <span id="clock" style="color:#718096;font-size:.85rem"></span>
  </div>
</div>
<div class="container-fluid p-4">
  <div class="d-flex justify-content-end mb-3">
    <select class="rng" id="range">
      <option value="1h">Last Hour</option>
      <option value="24h" selected>Last 24 Hours</option>
      <option value="7d">Last 7 Days</option>
      <option value="30d">Last 30 Days</option>
    </select>
  </div>
  <div class="row g-3 mb-2">
    <div class="col-md-3"><div class="card2"><div class="lbl">Total Attacks</div><div class="num"
id="total">—</div></div></div>
    <div class="col-md-3"><div class="card2 red"><div class="lbl">Unique Attackers</div><div
class="num red" id="unique">—</div></div></div>
    <div class="col-md-3"><div class="card2 green"><div class="lbl">Malware Samples</div><div
class="num green" id="malware">0</div></div></div>
    <div class="col-md-3"><div class="card2 yellow"><div class="lbl">Countries</div><div
class="num" style="color:#f7c94f" id="countries">—</div></div></div>
  </div>
  <div class="row g-3 mb-2">
    <div class="col-md-8"><div class="card2"><h6 style="color:#4f8ef7">Global Attack Map</h6><div
id="map"></div></div></div>
    <div class="col-md-4"><div class="card2"><h6 style="color:#4f8ef7">Attack
Timeline</h6><canvas id="tlChart" height="300"></canvas></div></div>
  </div>
  <div class="row g-3 mb-2">
    <div class="col-md-4"><div class="card2"><h6 style="color:#4f8ef7">Attack Types</h6><canvas
id="typeChart" height="250"></canvas></div></div>
    <div class="col-md-4"><div class="card2"><h6 style="color:#4f8ef7">Top Attacking IPs</h6><div
style="max-height:260px;overflow-y:auto"><table class="table table-
sm"><thead><tr><th>IP</th><th>Attacks</th><th></th></tr></thead><tbody
id="ips"></tbody></table></div></div></div>
    <div class="col-md-4"><div class="card2"><h6 style="color:#4f8ef7">Top Usernames</h6><div
style="max-height:260px;overflow-y:auto"><table class="table table-
sm"><thead><tr><th>Username</th><th>Count</th></tr></thead><tbody
id="users"></tbody></table></div></div></div>
  </div>
  <div class="card2"><h6 style="color:#4f8ef7">Real-time Attack Feed</h6><div id="feed"
style="max-height:300px;overflow-y:auto"><div style="color:#718096">Attacks ka wait ho raha
hai...</div></div></div>
</div>
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
<script>
```

```
const map=L.map('map').setView([20,0],2);
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png',{attribution:'© OpenStreetMap'}).addTo(map);
let markers=[],tl,tc;
async function loadStats(){
  const r=await fetch(`/api/stats?range=${document.getElementById('range').value}`);
  const d=await r.json();
  document.getElementById('total').textContent=d.total_attacks||0;
  document.getElementById('unique').textContent=d.unique_attackers||0;
  if(tl)tl.destroy();
  tl=new Chart(document.getElementById('tlChart').getContext('2d'),{type:'line',data:{labels:(d.timeline||[]).map(x=>new Date(x.timestamp).toLocaleTimeString()),datasets:[{label:'Attacks',data:(d.timeline||[]).map(x=>x.count),borderColor:'#4f8ef7',backgroundColor:'rgba(79,142,247,.15)',tension:.3,fill:true}]},options:{responsive:true,plugins:{legend:{display:false}},scales:{y:{beginAtZero:true,grid:{color:'#2d3250'},ticks:{color:'#718096'}},x:{grid:{display:false},ticks:{color:'#718096'}}}}});
  if(tc)tc.destroy();
  tc=new Chart(document.getElementById('typeChart').getContext('2d'),{type:'doughnut',data:{labels:(d.attack_types||[]).map(x=>x.name),datasets:[{data:(d.attack_types||[]).map(x=>x.count),backgroundColor:['#f74f4f','#f7c94f','#4ff7a0','#4f8ef7','#b44ff7'],borderWidth:0}]},options:{responsive:true,plugins:{legend:{position:'bottom',labels:{color:'#e2e8f0'}}}}});

document.getElementById('ips').innerHTML=(d.top_ips||[]).map(x=>`<tr><td><code>${x.ip}</code></td><td>${x.count}</td><td><a href="https://www.virustotal.com/gui/ip-address/${x.ip}" target="_blank" class="btn btn-sm btn-outline-secondary py-0">🔍</a></td></tr>`).join('');

document.getElementById('users').innerHTML=(d.top_usernames||[]).map(x=>`<tr><td>${x.username}</td><td>${x.count}</td></tr>`).join('');
}
async function loadFeed(){
  const r=await fetch('/api/recent_attacks');const d=await r.json();
  document.getElementById('feed').innerHTML='';
  d.forEach(a=>{
    const f=document.getElementById('feed');
    const el=document.createElement('div');el.className='feed-item';
    el.innerHTML=`<div class="d-flex justify-content-between"><span><b>${a.src_ip||'?'}</b> <span class="badge bg-secondary">${a.country||'Unknown'}</span> <span class="badge bg-danger">${a.attack_type||'unknown'}</span></span><small style="color:#718096">${new Date(a.timestamp).toLocaleTimeString()}</small></div>${a.username?`<small style="color:#718096">user: <b>${a.username}</b> | pass: <b>${a.password||'?'}</b></small>`:''}`;
    f.insertBefore(el,f.firstChild);
  });
}
document.addEventListener('DOMContentLoaded',()=>{
  loadStats();loadFeed();
  document.getElementById('range').addEventListener('change',()=>loadStats());
  setInterval(loadStats,30000);setInterval(loadFeed,15000);
  setInterval(()=>{document.getElementById('clock').textContent=new Date().toLocaleString();},1000);
});
```

```
</script>
</body>
</html>
```

```
# loadStats() JS
```

```
async function loadStats() {
  const r = await fetch('/api/stats?range=' +
    document.getElementById('range').value);
  const d = await r.json();
  document.getElementById('total').textContent = d.total_attacks || 0;
  document.getElementById('unique').textContent = d.unique_attackers || 0;
  document.getElementById('countries').textContent = d.countries || 0;
}
```

NOTE: Full dashboard.html conversation history mein hai

DAY 17 Build + ES Connection Fix

This configuration runs the dashboard container, connects it to Elasticsearch, exposes it on port 5000, and verifies system health using an API check.

STEP 3 Add Dashboard to docker-compose + Build

Goal: Service definition

```
# Add to docker-compose.yml
```

```
networks:
  honeypot_net:
    driver: bridge
    ipam:
      config:
        - subnet: 192.168.100.0/24
  monitoring_net:
    driver: bridge

services:
  cowrie:
    build: ./honeypot/cowrie
    container_name: cowrie_honeypot
    networks:
      honeypot_net:
        ipv4_address: 192.168.100.10
    ports:
      - "2222:2222"
```

```
      - "2223:2223"
    volumes:
      - ./data/logs/cowrie:/var/log/cowrie
    restart: unless-stopped

  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.6.0
    container_name: elasticsearch
    networks:
      - monitoring_net
    environment:
      - discovery.type=single-node
      - "ES_JAVA_OPTS=-Xms512m -Xmx512m"
      - xpack.security.enabled=false
    volumes:
      - es_data:/usr/share/elasticsearch/data
    ports:
      - "9200:9200"
    restart: unless-stopped

  logstash:
    image: docker.elastic.co/logstash/logstash:8.6.0
    container_name: logstash
    networks:
      - honeypot_net
      - monitoring_net
    volumes:
      - ./logging/logstash/pipeline:/usr/share/logstash/pipeline:ro
      - ./data/logs:/var/log/honeypot:ro
    depends_on:
      - elasticsearch
    restart: unless-stopped

  kibana:
    image: docker.elastic.co/kibana/kibana:8.6.0
    container_name: kibana
    networks:
      - monitoring_net
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
    ports:
      - "5601:5601"
    depends_on:
      - elasticsearch
    restart: unless-stopped
```

```
  filebeat:
    image: docker.elastic.co/beats/filebeat:8.6.0
    container_name: filebeat
    networks:
      - honeypot_net
      - monitoring_net
    user: root
    volumes:
      - ./logging/filebeat/filebeat.yml:/usr/share/filebeat/filebeat.yml:ro
      - ./data/logs:/var/log/honeypot:ro
      - /var/run/docker.sock:/var/run/docker.sock:ro
    depends_on:
      - logstash
    restart: unless-stopped

  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    networks:
      - monitoring_net
    volumes:
      - ./monitoring/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    networks:
      - monitoring_net
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin123
    ports:
      - "3000:3000"
    restart: unless-stopped

  dashboard:
    build: ./dashboard
    container_name: attack_dashboard
    networks:
      - monitoring_net
    ports:
      - "5000:5000"
    environment:
      - ELASTICSEARCH_HOST=http://elasticsearch:9200
```

```
      - SECRET_KEY=change-me-in-production
    volumes:
      - ./data/processed:/app/data/processed
    depends_on:
      - elasticsearch
    restart: unless-stopped

volumes:
  es_data:
```

```
# Build + start
docker compose down
docker compose up -d --build
docker compose ps
# Health check
sleep 10 && curl -s http://localhost:5000/api/health
```

Expected Output:

```
{"status":"healthy","elasticsearch":true}
```

These commands connect the dashboard container to Elasticsearch using its dynamic IP, run the container with required environment variables, and verify attack statistics through an API

STEP 4	Fix ES Connection (Direct IP Method)	**Goal:** Hostname nahi chalta — IP use karo

```
# Get ES IP
docker inspect elasticsearch | grep '"IPAddress"' | tail -1
# Restart dashboard with IP
docker rm -f attack_dashboard
docker run -d \
  --name attack_dashboard \
  --network healthcare-iot-honeypot_monitoring_net \
  -p 5000:5000 \
  -e ELASTICSEARCH_HOST=http://172.18.0.4:9200 \
  -e SECRET_KEY=change-me \
  -v $(pwd)/data/processed:/app/data/processed \
  healthcare-iot-honeypot-dashboard
# Verify
curl -s http://localhost:5000/api/stats | python3 -m json.tool
```

Expected Output:

```
{"total_attacks":95,"unique_attackers":10,"countries":11}
```

NOTE: 172.18.0.4 ki jagah actual ES IP daalo — har system restart ke baad IP change ho sakta hai

DAY 18 Map Data + Countries + Final Report

This code inserts sample attack and malware data into Elasticsearch to simulate real-world cyberattacks and visualize them on the dashboard map

STEP 5	Add Geolocation Data for World Map	**Goal:** 5 countries on map

```
# Use attacks alias
```

```
attacks

#  manually 5 add countries :
```

```
# Multiple countries manually
```

```
# Germany

curl -X POST "http://localhost:9200/honeypot-$(date +%Y.%m.%d)/_doc" -H 'Content-Type: application/json' -d '{"@timestamp":"'$(date -u +%Y-%m-%dT%H:%M:%S.000Z)'","attack_type":"bruteforce","src_ip":"80.82.0.1","geoip":{"country_name":"Germany","city_name":"Berlin","location":{"lat":52.5200,"lon":13.4050}}}'

# Japan

curl -X POST "http://localhost:9200/honeypot-$(date +%Y.%m.%d)/_doc" -H 'Content-Type: application/json' -d '{"@timestamp":"'$(date -u +%Y-%m-%dT%H:%M:%S.000Z)'","attack_type":"bruteforce","src_ip":"103.5.0.1","geoip":{"country_name":"Japan","city_name":"Tokyo","location":{"lat":35.6762,"lon":139.6503}}}'

# UK

curl -X POST "http://localhost:9200/honeypot-$(date +%Y.%m.%d)/_doc" -H 'Content-Type: application/json' -d '{"@timestamp":"'$(date -u +%Y-%m-%dT%H:%M:%S.000Z)'","attack_type":"credential_theft","src_ip":"185.0.0.1","geoip":{"country_name":"United Kingdom","city_name":"London","location":{"lat":51.5074,"lon":-0.1278}}}'
```

```
# Malware samples add
```

```
for i in 1 2 3; do

curl -s -X POST "http://localhost:9200/honeypot-$(date +%Y.%m.%d)/_doc" \

  -H 'Content-Type: application/json' \

  -d "{\"@timestamp\":\"$(date -u +%Y-%m-%dT%H:%M:%S.000Z)\",\"eventid\":\"cowrie.session.file_download\",\"attack_type\":\"malware_delivery\",\"src_ip\":\"45.142.0.$i\",\"shasum\":\"abc12$i\",\"filename\":\"payload$i.sh\",\"geoip\":{\"country_name\":\"Netherlands\",\"location\":{\"lat\":52.3676,\"lon\":4.9041}}}"

sleep 1

done
```

NOTE: Browser mein F5 press karo — map pe red dots dikhenge

These commands generate the final attack analysis report, verify output files, and ensure all system containers are running properly.

STEP 6	Final Report + Verify All	Goal: Project completion

```
# Run final report
source venv/bin/activate
python3 analysis/reports/generate_final_report.py
# Check all outputs
ls -lh analysis/reports/
ls -lh data/processed/
docker compose ps
```

```
# Restart dashboard
docker rm -f attack_dashboard
# Phir docker run with ES IP command
```

NOTE: Verify: curl -s http://localhost:5000/api/stats | python3 -m json.tool

NEW ADDITIONS / FIXES (Post-Deployment)
NEW ES connection fix: docker run with -e ELASTICSEARCH_HOST=http://ACTUAL_IP:9200 instead of hostname
NEW Countries count added to /api/stats: 'countries': {cardinality: {field: 'geoip.country_name.keyword'}}
NEW Countries count shown in dashboard: document.getElementById('countries').textContent = d.countries || 0
NEW Malware samples: add cowrie.session.file_download events with shasum field
NEW Real-time Attack Feed: attacks alias se fresh data -> F5 press karo -> feed mein entries aati hain
NEW start-honeypot.sh script: automatic ES IP detection + dashboard restart every time

Completion Checklist
✓ dashboard/Dockerfile created and builds successfully
✓ dashboard/app.py with /api/stats (includes countries count)
✓ dashboard/templates/dashboard.html dark theme created

- ✓ attack_dashboard container Up on port 5000
- ✓ curl http://localhost:5000/api/health returns elasticsearch: true
- ✓ Total Attacks and Unique Attackers numbers display correctly
- ✓ Attack Types chart shows bruteforce, command_injection, credential_theft
- ✓ Countries count shows correctly (not dash)
- ✓ World map shows red dots for 5+ countries
- ✓ Real-time Attack Feed shows entries after attacks alias
- ✓ analysis/reports/final_report_*.json generated

Week 4 — Required GitHub Commits

Commit Message	Files Changed
`git commit -m "feat: add Flask dashboard Dockerfile"`	`dashboard/Dockerfile`
`git commit -m "feat: add app.py with /api/stats + countries count"`	`dashboard/app.py`
`git commit -m "feat: add /api/recent_attacks and /api/health routes"`	`dashboard/app.py`
`git commit -m "feat: add dark theme dashboard.html with Chart.js"`	`dashboard/templates/dashboard.html`
`git commit -m "feat: add Leaflet.js global attack map"`	`dashboard/templates/dashboard.html`
`git commit -m "feat: add dashboard service to docker-compose.yml"`	`docker-compose.yml`
`git commit -m "fix: ES connection using direct IP instead of hostname"`	`dashboard/app.py, start-honeypot.sh`
`git commit -m "fix: add countries cardinality aggregation to stats API"`	`dashboard/app.py`
`git commit -m "feat: add malware delivery test events with shasum"`	`~/auto-attacks.sh`
`git commit -m "test: verify dashboard shows all 4 attack types"`	`http://localhost:5000`